

SEQUENTIAL SYSTEMS

The Complete Master Manual (Part I)

Course: EE/PC/B/T/225

Methodology: STC Sir FSM Synthesis Standard

Format: High-Fidelity Hand-Drawn Polygon

Vector Graphics

A definitive 6-Phase roadmap from foundational latch physics to complex Algorithmic State Machine (FSM) synthesis.

PHASE I: THE GENESIS OF MEMORY

COMBINATIONAL VS. SEQUENTIAL LOGIC

In combinational circuits (Adders, MUX), the output is strictly a function of the *present* inputs. If the inputs are removed, the output disappears. They possess zero memory.

Sequential circuits introduce a **Feedback Loop**. The output of the logic gates is routed back into the inputs. This physical loop "traps" the signal, allowing the circuit to remember its state even after the initial trigger is removed. A Sequential Circuit = Combinational Logic + Memory Elements.

1. The S-R Latch (Set-Reset)

The most fundamental memory element is the S-R Latch, typically constructed from two cross-coupled NOR or NAND gates.

- **Set (S=1, R=0):** Forces the output Q to 1.
- **Reset (S=0, R=1):** Forces the output Q to 0.
- **Hold (S=0, R=0):** The feedback loop sustains the previous state (Q_n).

- **Forbidden ($S=1, R=1$):** Forces both Q and $\bar{Q}$ to 0. This violates the complementary rule and creates an unpredictable race condition if both inputs drop to 0 simultaneously.

2. The Master-Slave JK Flip-Flop

To solve the SR forbidden state, the JK flip-flop introduces **Toggle Mode** ($J=1, K=1$). However, in a standard level-triggered latch, $J=1$ and $K=1$ causes a "Race-Around Condition" where the output oscillates uncontrollably while the clock is high.

THE MASTER-SLAVE SOLUTION

By cascading two latches and inverting the clock between them, the Master-Slave configuration breaks the continuous feedback loop. The Master accepts inputs when the clock is HIGH. The Slave updates the final output Q only when the clock transitions from HIGH to LOW (**Negative Edge Triggered**). This ensures the output toggles exactly once per clock cycle.

3. The Four Fundamental Excitation Tables

The Excitation Table is the most critical mathematical tool in sequential design. It answers the prescriptive question: *"What inputs must I apply to force the flip-flop to transition from Present State (Q_n) to Next State (Q_{n+1})?"*

State Transition		SR Flip-Flop		JK Flip-Flop		D Flip-Flop	T Flip-Flop
Q_n	Q_{n+1}	S	R	J	K	D	T
0	0	0	X	0	X	0	0
0	1	1	0	1	X	1	1
1	0	0	1	X	1	0	1
1	1	X	0	X	0	1	0

RULE OF THUMB MEMORIZATION

- **D-Flip Flop:** The easiest. D always exactly equals the Next State ($D = Q_{n+1}$).
- **T-Flip Flop:** $T=1$ if the state changes ($0 \rightarrow 1$ or $1 \rightarrow 0$). $T=0$ if it holds.
- **JK-Flip Flop:** The first column of J matches D. The second column of K matches the inverted D. The rest are "Don't Cares" (X).

PHASE II: HARDWARE COERCION (CONVERSIONS)

Hardware coercion (Flip-Flop Conversion) is the process of using combinational logic to trick one type of flip-flop into behaving exactly like another. This follows a strict 6-step algorithmic synthesis.

Case Study: Convert SR Flip-Flop to JK Flip-Flop

SYNTHESIS STRATEGY

1. Create the State Synthesis Table merging Target (JK) and Source (SR).
2. Extract Karnaugh Maps for S and R.
3. Draw the final logic circuit.

Target Inputs & State			Next State	Source Inputs	
J	K	Q_n	Q_{n+1}	S	R
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	X	0
1	1	0	1	1	0
1	1	1	0	0	1

K-Map for S

	KQ_n			
J	00	01	11	10
0	0	X	0	0
1	1	X	0	1

$$S = J \cdot \overline{Q}_n$$

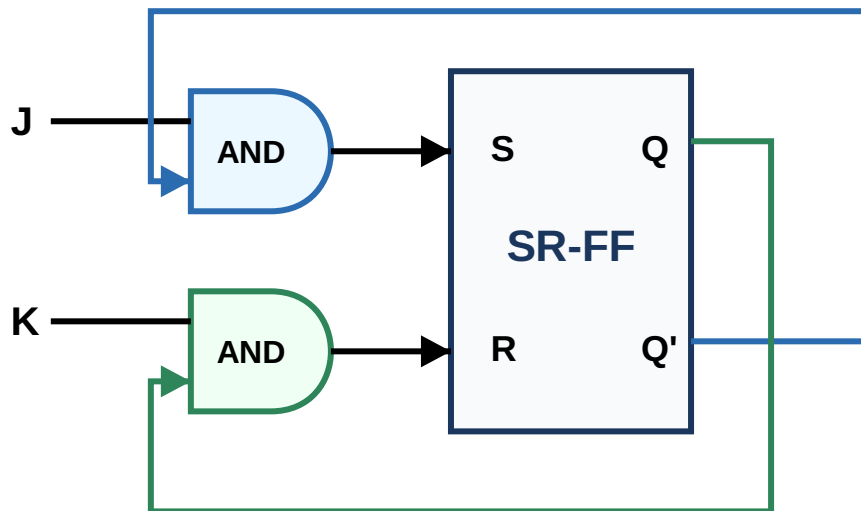
K-Map for R

	KQ_n			
J	00	01	11	10
0	X	0	1	X
1	0	0	1	0

$$R = K \cdot Q_n$$

Hardware Implementation (SR to JK)

Based on the derived K-Map logic, we wire the external J input through an AND gate with the $\bar{Q}$ feedback to generate S. We wire the external K input through an AND gate with the Q feedback to generate R. This perfectly synthesizes JK behavior.



PHASE III: ASYNCHRONOUS CASCADES (RIPPLE COUNTERS)

THE RIPPLE EFFECT

In an asynchronous counter, the master clock is applied **only** to the Least Significant Bit (LSB) flip-flop. The output of the LSB acts as the clock for the next flip-flop, and so on.

If we use negative-edge triggered T-Flip-Flops (with $T=1$ permanently), the next flip-flop toggles every time the previous flip-flop transitions from 1 to 0. This inherently creates a binary up-counter.

The Propagation Delay Crisis

Because the clock signal must literally "ripple" serially through the flip-flops, asynchronous counters suffer from cumulative propagation delay (t_{pd}). If a 4-bit counter uses flip-flops with a 20ns delay, the Most Significant Bit (MSB) will not update until 80ns after the clock pulse.

During this 80ns window, external decoding circuits will read temporary, invalid "garbage" states (Glitches). Therefore, ripple counters are unsuitable for high-frequency precision systems.

Mod-N Truncation (The Clear Override)

A natural 3-bit ripple counter counts from 0 to 7 (Mod-8). To design a Mod-6 counter (counting 0, 1, 2, 3, 4, 5), we must forcefully abort the sequence the moment it attempts to reach 6 (Binary 110).

We achieve this using a NAND gate connected to the asynchronous active-low $\overline{\text{CLR}}$ pins of all flip-flops. We wire the inputs of the NAND gate to Q_2 and Q_1 .

When the counter naturally hits state 6 ($Q_2=1$, $Q_1=1$, $Q_0=0$), the NAND gate outputs a 0. This active-low signal instantly activates the $\overline{\text{CLR}}$ pins, forcing all flip-flops back to 000 before the next clock pulse.

PHASE IV: SYNCHRONOUS ORCHESTRATION

THE PARALLEL SOLUTION

Synchronous counters eliminate propagation delay glitches. The master clock is wired simultaneously to every single flip-flop. State transitions are determined by complex combinational logic matrices feeding the input pins based on the current state.

Design a Mod-5 Synchronous Counter using D-Flip-Flops

- 1. State Allocation:** Mod-5 requires counting 0, 1, 2, 3, 4. Since $2^2 < 5 \leq 2^3$, we require 3 D-Flip-Flops (Q_2, Q_1, Q_0).
- 2. Excitation Matrix:** For a D-FF, $D = Q_{n+1}$. States 5, 6, and 7 are Unused (Don't Cares).

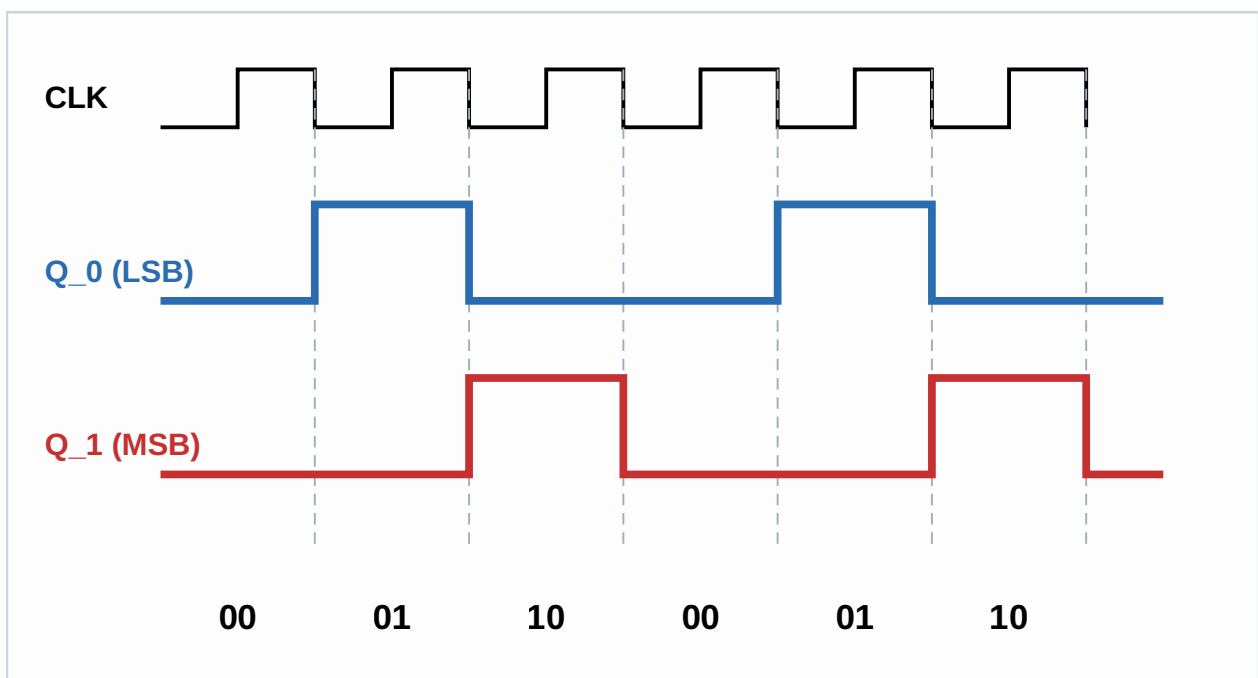
Present State			Next State / D-Inputs		
Q_2	Q_1	Q_0	D_2	D_1	D_0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	X	X	X
1	1	0	X	X	X
1	1	1	X	X	X

3. Minimized Equations (via K-Maps):

- $D_0 = \overline{Q}_2 \cdot \overline{Q}_0$
- $D_1 = Q_1 \oplus Q_0$
- $D_2 = Q_1 \cdot Q_0$

Timing Diagram Proof (Mod-3 Synchronous)

Consider a Mod-3 synchronous counter using JK flip-flops where $J_0 = \overline{Q}_1$, $K_0=1$ and $J_1=Q_0$, $K_1=1$. The diagram below rigorously proves the $00 \rightarrow 01 \rightarrow 10 \rightarrow 00$ sequence.



PHASE V: DATA CASCADES (SHIFT REGISTERS)

Registers are arrays of flip-flops used strictly for data storage and manipulation. While a basic register uses independent D-flip-flops to hold bits in parallel, a **Shift Register** cascades the flip-flops, connecting the Q output of one directly to the D input of the next.

The Universal Shift Register (IC 74194)

The 74194 is the pinnacle of shift register architecture. It utilizes an internal array of **4x1 Multiplexers** to dynamically alter the hardware routing into each flip-flop based on two Mode Control pins (S_1 , S_0).

S_1	S_0	Mode	Multiplexer Routing Logic
0	0	HOLD	MUX selects input 0. The Q output is routed straight back to the D input. The clock pulses, but state does not change.
0	1	SHIFT RIGHT	MUX selects input 1. Flip-flop receives data from its left-hand neighbor. ($Q_A \rightarrow Q_B$).
1	0	SHIFT LEFT	MUX selects input 2. Flip-flop receives data from its right-hand neighbor. ($Q_D \rightarrow Q_C$).
1	1	PARALLEL LOAD	MUX selects input 3. Flip-flop receives data directly from external parallel data pins.

PHASE VI: FINITE STATE MACHINE SYNTHESIS

MEALY VS. MOORE ARCHITECTURES

An FSM is a mathematical model of computation.

- **Moore Model:** The outputs are determined *strictly* by the current state. Outputs are written *inside* the state circles on a diagram. It is highly stable and synchronous.
- **Mealy Model:** The outputs are determined by the current state AND the current inputs. Outputs are written *on the transition arrows*. It requires fewer states but is susceptible to asynchronous input glitches.

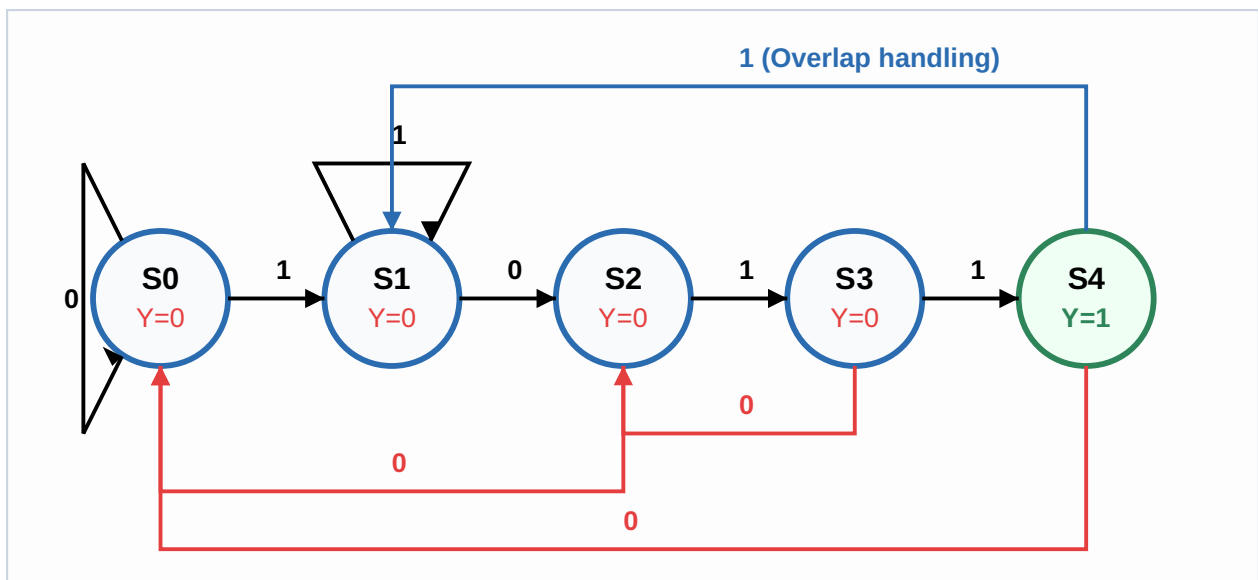
Synthesis Application: "1011" Sequence Detector (Moore Model)

Design an FSM that outputs a '1' only when the sequence 1, 0, 1, 1 is detected on a serial input line (X).

Step 1: State Definition

- **S0**: Reset State / Waiting for first '1'. ($Y=0$)
- **S1**: Detected "1". ($Y=0$)
- **S2**: Detected "10". ($Y=0$)
- **S3**: Detected "101". ($Y=0$)
- **S4**: Detected "1011". ($Y=1$) - *Success Node*

Step 2: Bulletproof State Diagram



END OF MASTER MANUAL

Sequential Systems - FSM Architecture Complete